Goupe 3 – Programmer-guide *SoundWay*

Esteban BARRACHO
Anas DAMLAKHI
Jonathan KONDOLI NKEBI
Arthur SMOOS
Bénédicte TUTEKA MUKUTA

10 novembre 2025

UNIVERSITÉ DE NAMUR

Table des matières

1 Preamble **3**

2 The chapters **4**

2.1 Where would I start if I had to continue developing this software ? 4

2.1.1 Where are the sources ? 4

2.1.2 Where's the documentation ? 5

2.1.3 Who were the developers ? 5

2.1.4 Where are the specifications ? 5

2.1.5 Who are the sponsors ? 5

2.1.6 What was the development environment like ? 6

2.1.7 How to run, deploy, test, etc. 6

2.1.8 The "keys" to architecture ! 8

2.2 Point of view 9

2.2.1 Top-down 9

2.3 Comment on the "motivation" behind my choices ! 10

2.3.1 Why things happen 10

2.4 Software libraries or components 11

2.4.1 Your users are developers 11

2.4.2 Examples of code : 11

2.5 APIs and interaction layers 13

2.5.1 BLE Advertising Interface 13

2.5.2 BLE Scanning Interface (Android) 13

2.5.3 Local Debug API 13

2.5.4 Authentication and User Session (Android) 13

2.6 Embedded code 14

2.6.1 Deployment therefore deserves special attention 14

2.6.2 Hardware Matters More. 14

2.7 Limitations and Future Work 15

2.7.1 Current technical limitations 15

2.7.2 Potential improvements 15

3 Glossary **16**

4 Bibliographie **17**

4.1 Further reading 17

4.2 Structures to compensate for pre-requisites 17

5 Index **17**

6 Annexes **18**

6.1 Annexe A : Future Plans 18

1 Preamble

This document provides a comprehensive programmer-oriented guide to **SoundWay**, a Bluetooth-based indoor navigation system developed in the framework of the course *INFOM435 – Interface Incarnée Augmentée* at the University of Namur (UNamur). The project explores embodied and augmented interaction through the combination of IoT hardware (Raspberry Pi) and a mobile Android application.

SoundWay connects **BLE beacons** running on a Raspberry Pi with a **Kotlin/Jetpack Compose** Android application. The Raspberry Pi acts as an emitter above doors or key points in a building, broadcasting Bluetooth Low Energy (BLE) signals. The mobile app works as a receiver that :

— detects and interprets BLE signals,
— estimates distance using RSSI calibration,
— displays the user's position on a map,
— provides voice guidance using Text-to-Speech (TTS),
— manages user accounts and profiles.

The SoundWay ecosystem is composed of three main technical bricks :

— a **Raspberry Pi emitter** (`Rasberry/`) that handles BLE advertising and signal calibration ;
— a **local debug API** (`ApiLocalADE/`) to simulate or log data and speed up development and testing ;
— an **Android receiver application** (package `com.example.soundway`) that integrates BLE scanning, map visualization and voice assistance.

This documentation is intended both for :

— developers who want to understand, maintain or extend the system (e.g. add new zones, new interaction modes or new buildings),
— teachers or researchers who want to reuse SoundWay as a reference implementation for embodied and augmented interaction labs.

The focus is put on :

— code structure and modularity,
— deployment and calibration procedures,
— design rationales behind the architecture and technical choices,
— limitations and possible future evolutions (multi-floor support, richer maps, accessibility improvements, etc.).

2 The chapters

2.1 Where would I start if I had to continue developing this software ?

2.1.1 Where are the sources ?

The sources are organized into several main components in the repository :

- `Rasberry/` : Python code and assets for the BLE emitter on Raspberry Pi.
- `ApiLocalADE/` : optional local debug API used for calibration, logging and experimenting with indoor positioning logic.
- **Android application project** : Kotlin/Jetpack Compose project (package `com.example.soundway`), usually opened with Android Studio.

From a methodological point of view, the project follows the classic triptych :

- **Analysis** : course requirements, constraints of indoor navigation, accessibility goals, and early sketches of user journeys.
- **Design** : interaction scenarios, placement of beacons, mapping between zones and BLE identifiers, UI wireframes for the Android app.
- **Implementation** : concrete source code in Python (BLE, calibration, optional API) and Kotlin (mobile app).

Rasberry/ (Raspberry Pi Emitter) This folder contains :

- `Rasberry.py` : main script to broadcast BLE signals on Linux/Raspberry Pi.

Android Application The Android receiver application is built with :

- **Kotlin** and **Jetpack Compose** for the UI,
- **Google Maps SDK** for map-based navigation,
- **Text-to-Speech (TTS)** for voice guidance,
- **BLE scanning** to detect beacons and compute positions.

It is available on Google Play under the package id `com.example.soundway`. In the codebase, it is organized as a classic Android Studio project with modules, composables and navigation graphs.

2.1.2 Where's the documentation ?

The primary documentation is :

— the **README** and markdown files located at the root of the repository (functional overview, quick start),
— this **programmer guide** in LaTeX, which details architecture, deployment and evolution paths,
— inline comments in the main scripts (`ble_advertiser.py`, `ble_scan_rssi.py`, Android Kotlin files).

2.1.3 Who were the developers ?

— Esteban BARRACHO
— Anas DAMLAKHI
— Jonathan KONDOLI NKEBI
— Arthur SMOOS
— Bénédicte TUTEKA MUKUTA

2.1.4 Where are the specifications ?

The original specifications are embedded in :

— the course sheet of *INFOM435 – Interface Incarnée Augmentée*,
— the project description and constraints provided by the teaching staff (indoor navigation, accessibility, embodied interaction),
— internal notes on use cases (guiding users inside a building, helping with orientation, enabling hands-free navigation).

These documents define the pedagogical objectives (exploring IoT + mobile interactions) and functional requirements (indoor navigation, BLE-based positioning, audio feedback).

2.1.5 Who are the sponsors ?

The **academic sponsor** of SoundWay is the course *INFOM435 – Interface Incarnée Augmentée* at the University of Namur. The project was carried out under the supervision of the teaching team and in the context of CSLabs, which provided the pedagogical and technical environment.

The goal was not only to deliver a working prototype, but also to reflect on embodied interaction, accessibility and the integration of IoT in real buildings.

2.1.6 What was the development environment like ?

Languages, editors, testing, continuous integration, etc. The SoundWay system relies on a mixed stack combining Python and Android technologies.

- **Programming languages :**
  - Python 3.12 for BLE emission, calibration and the optional local API.
  - Kotlin for the Android app, with Jetpack Compose for the UI layer.
- **Development environments :**
  - Visual Studio Code / PyCharm for the Raspberry Pi and local API scripts.
  - Android Studio for the mobile application.
- **Key libraries and SDKs :**
  - **Bleak** for cross-platform BLE on Raspberry Pi / Windows.
  - Standard Python libraries for logging, JSON handling and data analysis.
  - **Google Maps SDK**, **Text-to-Speech**, and Android BLE APIs for the mobile app.
- **Version control :** Git was used for code tracking, branches and collaboration, with the repository hosted on GitHub.
- **Testing :** Development relied mainly on :
  - empirical tests in real corridors and rooms,
  - RSSI calibration campaigns (`ble_scan_rssi.py`),
  - ad hoc logging via CSV files and debug consoles.

2.1.7 How to run, deploy, test, etc.

Raspberry Pi Emitter (Linux)

1. Install dependencies :

   ```
   sudo apt update
   sudo apt install python3 python3-venv python3-pip bluetooth bluez

   cd Rasberry
   python3 -m venv venv
   source venv/bin/activate
   pip install bleak
   ```

   This generates logs and updates `ble_calib.pkl`.
2. Start BLE broadcasting :

   ```
   python Rasberry.py
   ```

Android Receiver

1. Install the application from Google Play (package id `com.example.soundway`).
2. Create a user account inside the app.
3. Grant Bluetooth and Location permissions when requested.
4. Walk inside the instrumented building : the app detects beacons, displays your approximate position on the map and provides voice instructions.

Local Debug API To run the optional local API for debugging :

1. Go to `ApiLocalADE/`.
2. Create and activate a virtualenv if needed.
3. Install dependencies :

   ```
   pip install -r requirements.txt
   ```

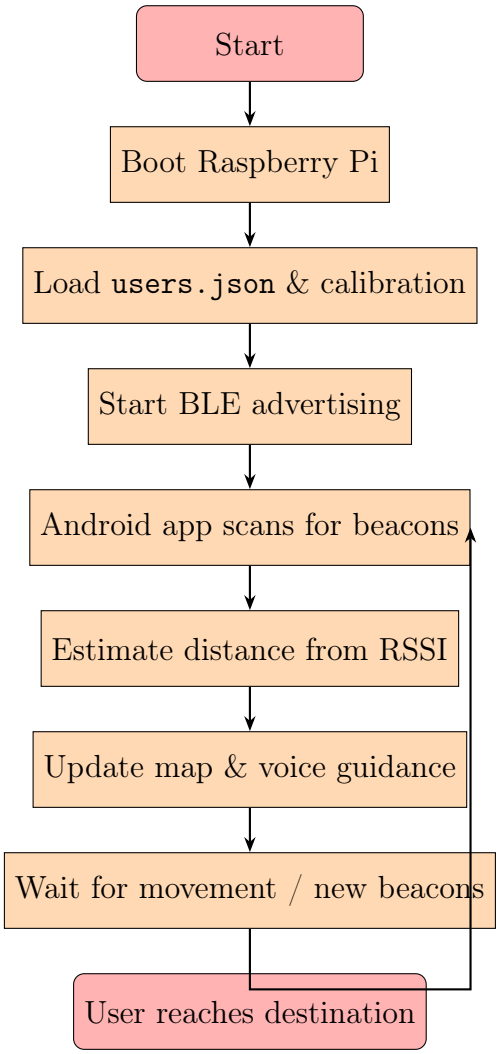
4. Run :

   ```
   python main.py
   ```

5. Use this API to replay logs, inspect RSSI behaviour or simulate beacons during Android development.

2.1.8 The "keys" to architecture !

Flowchart of the SoundWay runtime



Global architecture overview

- The **Raspberry Pi** acts as a BLE beacon emitter. It reads configuration from `users.json`, uses calibration data from `ble_calib.pkl`, and broadcasts identifiers for each zone.
- The **Android application** continuously scans for BLE beacons, filters them, estimates distances, and maps identifiers to zones displayed on a Google Map.
- The **local debug API** can be inserted between both to replay scenarios or study logs without physically moving in the building.

2.2 Point of view

2.2.1 Top-down

Responding to specific needs

The tree structure of modules and packages, with a motivation for each one The architecture was deliberately split according to responsibilities :

- `Rasberry/` :
  - `Rasberry.py` : core emitter logic, responsible for broadcasting BLE packets.
- **Android app (package `com.example.soundway`)** :
  - BLE layer : scanning, permissions, device filtering.
  - Domain layer : zone objects, distance estimation, navigation logic.
  - UI layer : Jetpack Compose screens, map view, voice control.
  - Account layer : user registration, login, profile management.

The main components of the architecture

- **BLE Emitter** :
  - runs continuously on the Raspberry Pi ;
  - encodes the zone identifier in advertising packets ;
  - relies on precomputed calibration to help distance estimation.
- **Receiver + Navigation Engine (Android)** :
  - collects RSSI values for each detected beacon ;
  - estimates the user's zone and approximate distance ;
  - updates map and triggers voice instructions.
- **Debug & Calibration Tools** :
  - log fine-grained RSSI traces ;
  - help adjust thresholds and zones ;
  - simulate or replay navigation without a live emitter.

Documentation on the application's run-time behaviour At run-time, several behaviours are important :

- **Dynamic calibration** : RSSI values are noisy. Logs and calibration tools help refine distance thresholds over time.
- **Permission management** : the Android app must react gracefully if Bluetooth or Location permissions are disabled, by explaining to the user what to do.
- **Energy impact** : BLE scanning and map rendering can consume battery ; scanning intervals and filters are tuned accordingly.
- **Resilience** : if no beacon is detected, the app should not crash but instead inform the user that they are out of range or in a non-instrumented area.

2.3 Comment on the "motivation" behind my choices !

2.3.1 Why things happen

The main design choices in SoundWay follow three axes :

1. **Embodied interaction and accessibility** :
   — Use of voice guidance (TTS) to help users who cannot rely solely on visual cues.
   — Simple, high-contrast UI through Jetpack Compose for readability.
   — Hands-free interaction : the user can keep the phone in a pocket and follow audio instructions.
2. **Simplicity and robustness on the hardware side** :
   — Use of a Raspberry Pi and BLE beacons : inexpensive, easy to reinstall, and compatible with many environments.
   — Python and Bleak chosen for their portability and maturity with BLE.
   — Calibration and logs stored in simple formats (`pkl`, `csv`) to allow quick iteration.
3. **Pedagogical value and extensibility** :
   — Clear separation between emitter, receiver and API tools, to allow students to work on different components independently.
   — Use of modern Android stack (Kotlin + Jetpack Compose) to align with industry practices.
   — Architecture that can be easily extended : multi-beacon triangulation, multi-floor support, integration of indoor maps, etc.

2.4 Software libraries or components

2.4.1 Your users are developers

For developers, the most important technical components are :

— **Python 3.12** : main language for the emitter and calibration tools.
— **Bleak** : cross-platform BLE library used for advertising and scanning.
— **Android SDK 26+** : minimum supported Android version for the app.
— **Kotlin + Jetpack Compose** : used for building the UI and navigation flows.
— **Google Maps SDK** : displays zones and the approximate user position on a map.
— **Text-to-Speech (TTS)** : embedded Android service for voice guidance.

2.4.2 Examples of code :

Increasing complexity Basic example (Python) : starting BLE advertising

Commentaire : ce code illustre le squelette typique de `ble_advertiser.py` : création d'un client BLE, configuration et démarrage de l'émission.

```
from bleak import BleakAdvertiser

BEACON_UUID = "0000a001-0000-1000-8000-00805f9b34fb"
ZONE_ID = 1

async def main():
    async with BleakAdvertiser() as advertiser:
        payload = build_beacon_payload(BEACON_UUID, ZONE_ID)
        await advertiser.start(payload)
        print("Advertising started for zone", ZONE_ID)
        await asyncio.Event().wait()  # Run until interrupted

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())
```

Intermediate example (Python) : logging RSSI during calibration

Commentaire : ce fragment montre comment enregistrer les valeurs RSSI dans un fichier CSV pour pouvoir ensuite ajuster les modèles de distance.

```
import csv
from datetime import datetime

def log_rssi(device_id: str, rssi: int, csv_path: str = "ble_multi_rssi_log.csv"):
    with open(csv_path, mode="a", newline="") as f:
        writer = csv.writer(f)
        writer.writerow([datetime.now().isoformat(), device_id, rssi])
```

Advanced example (Kotlin) : reacting to beacon detection in the Android app

Commentaire : ce pseudo-exemple Kotlin illustre comment la détection d'un beacon déclenche la mise à jour de la zone et de la navigation.

```
// Pseudo-Kotlin for illustration
fun onBeaconDetected(beaconId: String, rssi: Int) {
    val zone = zoneRepository.resolveZone(beaconId)
    val distance = distanceEstimator.estimate(rssi, zone.calibration)
    navigationState.updatePosition(zone, distance)
    tts.speak("Vous êtes proche de la zone ${zone.label}")
}
```

2.5 APIs and interaction layers

While SoundWay is primarily based on BLE rather than HTTP APIs, several interaction layers play a similar role for developers.

2.5.1 BLE Advertising Interface

The BLE advertising interface defines :

— the **payload structure** (UUID, zone identifier, optional flags),
— the **advertising interval** (trade-off between responsiveness and energy),
— the mapping between **zone identifiers** and physical locations (stored in `users.json`).

2.5.2 BLE Scanning Interface (Android)

On the Android side :

— the scanning layer filters by UUID and signal strength,
— only relevant beacons are passed to the navigation layer,
— debouncing and smoothing are applied to avoid noisy jumps.

2.5.3 Local Debug API

The optional local API (`ApiLocalADE/`) exposes endpoints such as :

— `/logs` : return a subset of recorded RSSI logs,
— `/zones` : return the list of known zones and their identifiers,
— `/simulate` : send fake beacon detections to the Android app for UI testing.

2.5.4 Authentication and User Session (Android)

The Android app includes :

— a basic account system (registration, login),
— local persistence of user preferences (preferred language, voice speed, etc.),
— optional server-side account management in future versions.

2.6 Embedded code

2.6.1 Deployment therefore deserves special attention

Although the core components are relatively lightweight, deployment involves hardware, OS configuration and mobile distribution.

- **Installing Python dependencies** :
  - use `python3 -m venv venv` and `pip install -r requirements.txt` where appropriate ;
  - on Raspberry Pi, ensure `bluetooth` and `bluez` are installed.
- **Configuring the Raspberry Pi** :
  - enable Bluetooth in `raspi-config` ;
  - make sure the emitter script is started at boot (e.g. via `systemd` service).
- **Android deployment** :
  - the app is signed and deployed through Google Play ;
  - for development, use Android Studio and USB debugging.
- **Log management** :
  - logs on Raspberry Pi (`ble_multi_rssi_log.csv`) are essential for understanding signal behaviour ;
  - Android logs (logcat) help to debug permission issues or BLE failures.

2.6.2 Hardware Matters More...

Since SoundWay is literally embodied in hardware and physical space, hardware choices have a direct impact on quality :

- **Raspberry Pi model** :
  - models with built-in BLE (e.g. Pi 3, Pi 4) are recommended ;
  - external USB dongles can be used but must be tested thoroughly.
- **Placement of beacons** :
  - beacons should be placed at consistent heights (e.g. above doors) ;
  - metal structures and thick walls strongly affect signal propagation.
- **Android phone characteristics** :
  - BLE antenna quality and Android BLE implementation differ between devices ;
  - it is useful to test with several phone models when calibrating.

2.7 Limitations and Future Work

2.7.1 Current technical limitations

- **RSSI instability** : BLE signal strength is highly variable, especially when users move or when there are many obstacles. Distance estimation remains approximate.
- **Single-beacon logic** : the current prototype mostly relies on the nearest beacon rather than full multi-beacon triangulation.
- **Static maps** : maps are pre-defined for one building ; extending to multiple floors or complex layouts would require additional work.
- **Limited test automation** : tests are mainly manual and scenario-based ; there is no large automated test suite for regression.
- **Offline behaviour** : if Google Maps is not available or if permissions are denied, the app offers limited functionality.

2.7.2 Potential improvements

- **Triangulation and multi-beacon fusion** : combine signals from several beacons to improve accuracy, possibly with Kalman filters or particle filters.
- **Advanced calibration** : build per-building, per-zone calibration profiles and adapt thresholds dynamically.
- **Indoor map integration** : integrate precise indoor maps (with corridors, stairs, elevators) and compute optimal paths.
- **Accessibility enhancements** : add haptic feedback, more detailed voice instructions, multilingual support and configuration settings.
- **Cloud backend** : allow synchronisation of user preferences, analytics on usage, and remote configuration of zones.
- **Automated testing** : add unit tests, integration tests and replay of recorded calibration sessions for non-regression.

3 Glossary

BLE (Bluetooth Low Energy) Wireless communication technology optimized for low-power, short-range communication. Used here for indoor positioning.

Beacon A device or script that periodically broadcasts BLE packets containing identifiers used for localization.

RSSI Received Signal Strength Indicator : a measure of the power present in a received radio signal. Used as a proxy for distance.

Raspberry Pi A small, affordable single-board computer used in this project to run the BLE emitter.

Bleak A Python BLE library that provides a simple API for scanning and advertising on multiple platforms.

Jetpack Compose Modern UI toolkit for building native Android interfaces in Kotlin using a declarative paradigm.

Text-to-Speech (TTS) Service that converts text into spoken voice output, enabling audio guidance for users.

Zone A logical area inside the building (e.g. corridor, room, entrance) associated with one or more beacons.

Calibration Process of measuring RSSI at known distances in order to build or adjust a model that maps signal strength to distance.

Indoor Navigation The process of guiding users inside buildings where GPS is unreliable or unavailable.

ApiLocalADE Optional local HTTP API used in SoundWay for debugging, log replay and simulation.

INFOM435 Course *Interface Incarnée Augmentée* at UNamur, under which SoundWay was developed.

4 Bibliographie

4.1 Further reading

Références

[1] Robin Heydon. *Bluetooth Low Energy : The Developer's Handbook*. Prentice Hall, 2013.

[2] Theodore S. Rappaport. *Wireless Communications : Principles and Practice*. Second edition, Prentice Hall, 2002.

[3] Bill Phillips, Chris Stewart, Kristin Marsicano. *Android Programming : The Big Nerd Ranch Guide*. Fourth edition, Big Nerd Ranch Guides, 2019.

[4] Sergey Vasiliev. *Jetpack Compose by Example*. Leanpub, 2022.

4.2 Structures to compensate for pre-requisites

Références

[1] David Beazley and Brian K. Jones. *Python Cookbook*. Third edition, O'Reilly Media, 2013.

[2] Z. Farid, R. Nordin, M. Ismail. "Recent advances in wireless indoor localization techniques and system." *Journal of Computer Networks and Communications*, 2013.

[3] Sarah Horton and Whitney Quesenbery. *A Web for Everyone : Designing Accessible User Experiences*. Rosenfeld Media, 2014.

5 Index

API, 13

Introduction, 3

Software features, 3

6 Annexes

6.1 Annexe A : Future Plans

In future iterations, SoundWay could evolve towards a more integrated indoor navigation platform :

— **Multi-building support** : configure several buildings with their own sets of beacons, maps and calibration profiles.
— **Dynamic content** : associate audio descriptions, alerts or contextual help with specific zones (e.g. elevators, exits, reception).
— **Advanced analytics** : record anonymized trajectories (with consent) to optimize beacon placement and detect recurrent navigation issues.
— **Integration with other systems** : link SoundWay with room booking systems, emergency evacuation plans, or campus information services.

Such extensions would preserve the core philosophy of the project : lightweight hardware, accessible interaction, and modular code that can be reused in other pedagogical or real-world contexts.

© 2025 Esteban BARRACHO, Anas DAMLAKHI, Jonathan KONDOLI NKEBI,
Arthur SMOOS, Bénédicte TUTEKA MUKUTA.
This document is distributed under the terms of the Creative Commons CC BY-ND (Attribution-No Derivative Works) licence.

This work is licensed under a Creative Commons “Attribution-NoDerivatives 4.0 International” license.